

Apresentação do Projeto: Abastece Manaus

1. Visão geral

O Abastece Manaus é um aplicativo mobile/web feito para ajudar motoristas de Manaus a encontrar postos de combustível, comparar preços e contribuir com informações atualizadas para a comunidade.

A proposta principal é simples:

- mostrar postos no mapa;
- listar postos por preço, distância ou avaliação;
- comparar dois postos lado a lado;
- salvar postos favoritos;
- permitir que usuários reportem novos preços;
- criar alertas de preço;
- manter uma experiência parecida no Android, iOS e Web usando Expo.

O app usa uma base local de postos de Manaus em `data/stations.ts`, com nome, bandeira, endereço, bairro, coordenadas, combustíveis, preços, histórico, avaliações e serviços disponíveis.

2. Tecnologias usadas

O projeto foi construído com:

- Expo: ambiente principal para rodar React Native em Android, iOS e Web.
- React Native: criação das telas e componentes visuais.
- Expo Router: navegação por arquivos, como `app/(tabs)/index.tsx`, `app/station/[id].tsx` e `app/report/[id].tsx`.
- TypeScript: tipagem dos dados, telas e regras.
- React Context + Reducer: estado global do aplicativo em `context/AppContext.tsx`.
- AsyncStorage: persistência local de favoritos, alertas, contribuições, filtros e comparadores.
- Expo Location: captura da localização do usuário no app nativo.
- React Native Maps: mapa interativo no Android/iOS.
- tRPC + React Query: camada de API tipada entre frontend e backend.
- Express: servidor backend.
- Drizzle ORM + SQLite: banco para usuários, perfis OAuth e contribuições.
- Vitest: testes automatizados.

3. Como rodar o projeto

Pré-requisitos:

- Node.js 22;
- pnpm via Corepack;
- arquivo `.env` configurado.

Comandos principais:

```
pnpm install
pnpm run dev
```

O comando `pnpm run dev` sobe duas partes ao mesmo tempo:

- `dev:server`: inicia o backend em `server/_core/index.ts`;

- dev:metro: inicia o Expo Metro Bundler.

Para testar:

- no celular: usar Expo Go e ler o QR Code;
- no navegador: pressionar w no terminal do Expo;
- Android/iOS emulador: usar a ou i, quando o ambiente estiver configurado.

4. Estrutura principal do projeto

```
app/  
_layout.tsx Configuração global do app, providers e rotas  
login.tsx Tela de login  
oauth/callback.tsx Retorno do login OAuth  
(tabs)/  
_layout.tsx Abas principais protegidas por login  
index.tsx Tela do mapa  
list.tsx Tela de listagem dos postos  
compare.tsx Comparação entre dois postos  
favorites.tsx Favoritos  
profile.tsx Perfil, alertas e configurações  
station/[id].tsx Detalhes de um posto  
report/[id].tsx Reporte de preço  
components/  
StationCard.tsx Card reutilizável de posto  
FuelTypeFilter.tsx Filtro por combustível  
map/ Mapa nativo e fallback web  
context/  
AppContext.tsx Estado global do app  
data/  
stations.ts Base de postos e funções de cálculo  
server/  
routers.ts Rotas tRPC  
db.ts Acesso ao banco  
_core/ Infraestrutura do servidor  
drizzle/  
schema.ts Tabelas do banco
```

5. Fluxo de navegação

O app começa em app/_layout.tsx, onde são configurados os providers globais:

- tema;
- safe area;
- React Query;
- tRPC;
- AppProvider;
- pilha de navegação do Expo Router.

Depois, o usuário passa pela tela de login em app/login.tsx.

A área principal fica em app/(tabs)/_layout.tsx. Esse arquivo verifica se existe state.user.

Se não existir, redireciona para /login. Quando existe usuário, libera as abas:

- Mapa
- Postos
- Comparar
- Favoritos
- Perfil

Também existe a opção Entrar sem login, que cria um usuário demo no estado global para

facilitar apresentação e testes.

6. Estado global do aplicativo

O estado central fica em `context/AppContext.tsx`.

Ele guarda:

- lista de postos;
- IDs dos favoritos;
- alertas de preço;
- contribuições do usuário;
- combustível selecionado;
- tipo de ordenação;
- distância máxima do filtro;
- localização do usuário;
- postos selecionados para comparação;
- usuário autenticado.

As ações principais são:

- `TOGGLE_FAVORITE`: adiciona ou remove um posto dos favoritos.
- `REPORT_PRICE`: atualiza o preço de um combustível e registra uma contribuição.
- `ADD_ALERT`: cria alerta de preço.
- `TOGGLE_ALERT`: ativa/desativa alerta.
- `DELETE_ALERT`: remove alerta.
- `SET_FUEL_TYPE`: troca o combustível selecionado.
- `SET_SORT`: muda a ordenação.
- `SET_MAX_DISTANCE`: altera o raio de busca.
- `SET_USER_LOCATION`: salva a localização atual.
- `SET_COMPARATOR_IDS`: define os dois postos comparados.
- `SET_USER`: salva o usuário logado ou demo.

Parte desse estado é persistida no `AsyncStorage`. Assim, favoritos, alertas, contribuições e filtros continuam salvos depois que o app fecha.

7. Base de dados dos postos

O arquivo `data/stations.ts` é a base principal do domínio.

Ele define:

- `FuelType`: tipos de combustível (gasolina, aditivada, etanol, diesel, gnv);
- `FuelPrice`: preço, data de atualização e quantidade de confirmações;
- `Station`: estrutura completa do posto;
- `STATIONS`: lista de postos;
- funções de apoio.

Funções importantes:

- `getPriceCategory(price)`: classifica o preço como barato, médio ou caro.
- `getPriceCategoryColor(category)`: retorna a cor usada no mapa e nos cards.
- `isOutdated(date)`: marca preço como desatualizado após 48 horas.
- `getPriceConfidence(price)`: classifica preço como confirmado, recente ou desatualizado.
- `formatTimeAgo(date)`: mostra textos como "2h atrás" ou "3d atrás".
- `calculateDistance(...)`: calcula distância em km entre usuário e posto.
- `inferNeighborhood(...)`: tenta inferir o bairro quando o dado vem genérico.

8. Tela de mapa

Arquivo: `app/(tabs)/index.tsx`

Essa é a tela principal do app.

Ela mostra:

- mapa centralizado em Manaus;
- filtro por tipo de combustível;
- marcadores de postos;
- preço direto no marcador;
- cor do marcador conforme categoria de preço;
- botão para centralizar na localização do usuário;
- legenda de barato, médio e caro;
- resumo do posto selecionado em um painel inferior.

No Android/iOS, o mapa real vem de `components/map/index.tsx` usando `react-native-maps`.

Na Web, existe um fallback em `components/map/index.web.tsx`, porque o mapa interativo foi deixado disponível apenas para app nativo.

9. Tela de listagem

Arquivo: `app/(tabs)/list.tsx`

A tela de postos permite encontrar postos de forma mais detalhada.

Recursos:

- busca por nome, bairro, bandeira ou endereço;
- filtro por combustível;
- filtro por raio de distância;
- opção de mostrar todos os postos;
- filtro para mostrar somente preços confirmados;
- ordenação por preço, distância ou avaliação;
- contagem de resultados;
- renderização otimizada com `FlatList`.

Cada posto é exibido com `StationCard`, mostrando:

- nome;
- bairro/endereço;
- preço do combustível selecionado;
- distância;
- tempo desde atualização;
- número de confirmações;
- selo de confiança;
- botão de favorito.

10. Detalhes do posto

Arquivo: `app/station/[id].tsx`

Quando o usuário toca em um posto, abre a tela de detalhes.

Ela mostra:

- nome e bandeira;
- endereço e bairro;

- distância até o usuário;
- serviços disponíveis;
- lista de preços por combustível;
- status de confiança do preço;
- histórico em gráfico quando existir histórico suficiente;
- avaliações, quando cadastradas;
- botão para favoritar;
- botão para reportar preço;
- botão para abrir rota no app de mapas.

O botão Como Chegar abre:

- Google Maps na Web;
- Apple Maps no iOS;
- intent geo: no Android, com fallback para Google Maps.

11. Reporte de preço

Arquivo: app/report/[id].tsx

Essa tela permite o usuário contribuir informando um novo preço.

Fluxo:

- Usuário escolhe o tipo de combustível.
- Digita o preço por litro.
- O app valida se o preço está entre R\$ 1,00 e R\$ 20,00.
- Ao confirmar, dispara a ação REPORT_PRICE.
- O preço do posto é atualizado no estado local.
- A contribuição aparece no perfil do usuário.

Hoje o fluxo principal de reporte atualiza o estado local do app. O backend também tem rotas para contribuições, mas a tela atual está usando o AppContext diretamente.

12. Comparação de postos

Arquivo: app/(tabs)/compare.tsx

A tela de comparação permite escolher dois postos e comparar:

- preço do combustível selecionado;
- diferença por litro;
- economia estimada para um tanque;
- distância;
- avaliação.

O tamanho padrão do tanque é 50 litros, mas o usuário pode alterar. A economia é calculada assim:

$$\text{economia} = \text{diferença_por_litro} * \text{capacidade_do_tanque}$$

Exemplo:

Posto A: R\$ 5,90
 Posto B: R\$ 6,20
 Diferença: R\$ 0,30 por litro
 Tanque: 50 L
 Economia estimada: R\$ 15,00

13. Favoritos

Arquivo: `app/(tabs)/favorites.tsx`

A tela de favoritos mostra todos os postos salvos pelo usuário.

Recursos:

- lista de postos favoritos;
- distância calculada até cada favorito;
- opção de limpar todos;
- mensagem vazia quando não há favoritos;
- indicação de variação de preço quando existe histórico suficiente.

Favoritar um posto é feito por `TOGGLE_FAVORITE`, tanto no card quanto na tela de detalhes.

14. Perfil, alertas e tema

Arquivo: `app/(tabs)/profile.tsx`

A tela de perfil concentra:

- identificação do usuário;
- número de contribuições;
- criação de alertas de preço;
- ativar/desativar alerta;
- excluir alerta;
- contribuições recentes;
- alternância de tema claro/escuro;
- informações do app.

Um alerta tem:

- combustível;
- preço máximo;
- posto opcional;
- status ativo/inativo;
- data de criação.

No estado atual, os alertas são cadastrados e persistidos localmente. A lógica de notificação real pode ser uma evolução futura.

15. Backend e banco de dados

O backend fica em `server/`.

Arquivo principal de rotas: `server/routers.ts`.

Rotas existentes:

- `auth.me`: retorna usuário autenticado.
- `auth.logout`: encerra sessão limpando cookie.
- `oauth.getOrCreateUser`: cria ou recupera usuário OAuth.
- `oauth.getProfile`: busca perfil OAuth do usuário autenticado.
- `contributions.list`: lista contribuições do usuário logado.
- `contributions.add`: salva uma contribuição.
- `contributions.byStation`: lista contribuições de um posto.

O banco é definido em `drizzle/schema.ts`.

Tabelas:

- `users`: usuários da aplicação.
- `userProfiles`: dados de provedor OAuth.

- contributions: histórico de preços enviados pelos usuários.

O acesso ao banco acontece em server/db.ts, usando Drizzle com SQLite via better-sqlite3.

16. Autenticação

O app tem autenticação com OAuth.

Arquivos principais:

- app/login.tsx: tela com login Google/Facebook e entrada demo.
- app/oauth/callback.tsx: processa o retorno do OAuth.
- hooks/use-auth.ts: utilitário para buscar usuário, validar sessão e fazer logout.
- constants/oauth.ts: configuração dos provedores e URLs.
- lib/_core/auth.ts: armazenamento de token e dados do usuário.

Na Web, o fluxo usa cookie de sessão. No app nativo, usa token armazenado localmente.

Para apresentação, a opção Entrar sem login é útil porque evita depender de credenciais OAuth durante a demonstração.

17. Regras de negócio principais

Classificação de preço

Em getPriceCategory:

- até 6.05: barato;
- até 6.30: médio;
- acima disso: caro.

Essas categorias mudam as cores dos marcadores e dos badges de preço.

Confiança do preço

Em getPriceConfidence:

- confirmed: atualizado até 24 horas e com pelo menos 2 confirmações;
- recent: atualizado até 48 horas;
- outdated: mais de 48 horas.

Isso ajuda o usuário a saber se o preço é confiável.

Distância

calculateDistance usa fórmula geográfica para calcular distância aproximada entre dois pontos por latitude e longitude.

Quando o usuário não tem localização disponível, o app usa MANAUS_CENTER como referência.

18. Limitações atuais

Pontos importantes para explicar com transparência:

- A base de postos vem de data/stations.ts, não de uma API externa em tempo real.
- O mapa interativo completo está disponível no Android/iOS; na Web há uma tela informativa.
- Alertas são salvos localmente, mas ainda não disparam notificações reais.
- Reporte de preço atualiza o estado local na interface; o backend possui rotas de contribuição para uma integração mais completa.
- Alguns dados como avaliações e histórico podem estar vazios em muitos postos.

- Se o usuário limpar o armazenamento local, favoritos, alertas e contribuições locais são perdidos.

19. Roteiro sugerido para apresentar à equipe

Abertura

"O Abastece Manaus é um aplicativo para ajudar motoristas a encontrar combustível mais barato em Manaus, comparar postos e contribuir com preços atualizados."

Demonstração prática

- Entrar no app usando Entrar sem login.
- Mostrar a aba Mapa e explicar as cores dos preços.
- Trocar o tipo de combustível no filtro.
- Tocar em um posto e abrir detalhes.
- Mostrar endereço, distância, preços, confiança e botão de rota.
- Reportar um preço fictício.
- Ir para Postos e demonstrar busca, ordenação e filtros.
- Ir para Comparar e selecionar dois postos.
- Mostrar cálculo de economia por tanque.
- Favoritar um posto e mostrar em Favoritos.
- Abrir Perfil, criar alerta e mostrar contribuições.

Parte técnica

"Tecnicamente, o app usa Expo com React Native. A navegação é feita pelo Expo Router. O estado global fica no AppContext e é salvo no AsyncStorage. A base de postos e as regras de preço ficam em data/stations.ts. O backend usa tRPC, Express, Drizzle e SQLite para autenticação e contribuições."

Fechamento

"O projeto já entrega uma experiência funcional para busca, comparação e colaboração. As próximas evoluções naturais seriam integrar os reportes diretamente ao backend em todas as telas, ativar notificações reais para alertas e substituir a base local por uma fonte dinâmica de preços."

20. Possíveis perguntas da equipe

Os preços são em tempo real"

Ainda não. O app usa uma base local e contribuições no estado da aplicação. A estrutura de backend já permite evoluir para salvar e consultar contribuições reais.

Funciona na Web"

Sim, a interface roda na Web. Porém, o mapa interativo completo foi implementado para Android/iOS; na Web existe um fallback informativo.

O usuário precisa fazer login"

O app tem login OAuth, mas também tem entrada demo para apresentação e testes.

Como o app sabe a distância"

Ele usa a localização do usuário via Expo Location no app nativo. Se não houver permissão, usa o centro de Manaus como referência.

Como um preço vira confirmado"

Pela regra atual, precisa ter até 24 horas e pelo menos 2 confirmações.

Onde os favoritos ficam salvos"

No AsyncStorage do dispositivo/navegador.

Qual seria o próximo passo técnico"

Conectar a tela de reporte ao endpoint `contributions.add`, carregar contribuições por posto com `contributions.byStation` e usar esses dados para atualizar preços de forma compartilhada entre usuários.